Michael Pham-Tran, Evan Lara

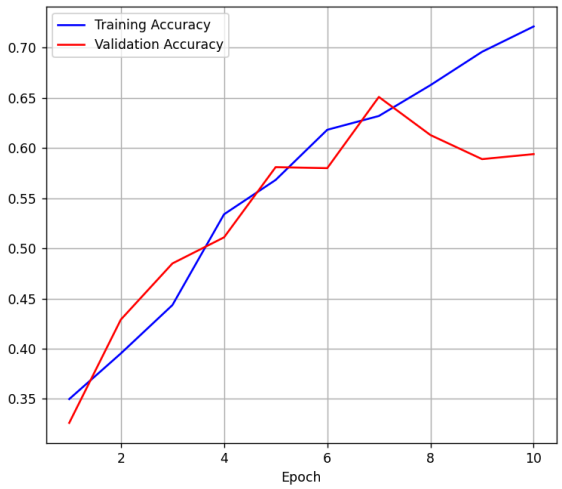
Project 6 Report

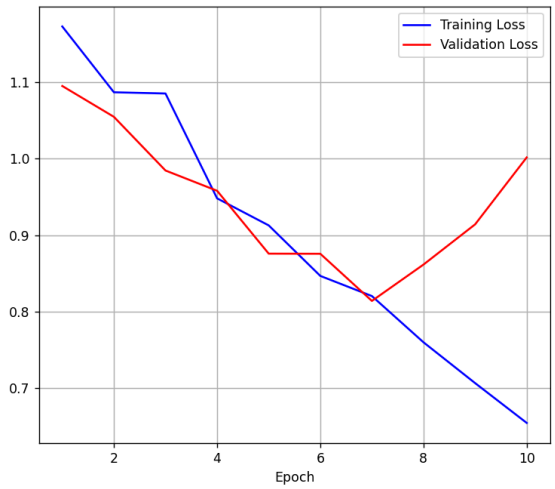
Section 5

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 62, 62, 32)	896
batch_normalization (BN)	(None, 62, 62, 32)	128
max_pooling2d (MaxPool2D)	(None, 31, 31, 32)	0
conv2d_1 (Conv2D)	(None, 29, 29, 64)	18496
batch_normalization_1	(None, 29, 29, 64)	256
max_pooling2d_1	(None, 14, 14, 64)	0
conv2d_2 (Conv2D)	(None, 12, 12, 128)	73856
batch_normalization_2	(None, 12, 12, 128)	512
max_pooling2d_2	(None, 6, 6, 128)	0
flatten (Flatten)	(None, 4608)	0
dense (Dense)	(None, 128)	589952
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 684,483
Trainable params: 683,843
Non-trainable params: 640





The training and validation accuracy seems to increase as epoch increases, but drops after epoch 7. The training and validation loss decreases as epoch increases. However the loss seems to increase after epoch 7. This odd change in direction might be caused by overfitting.

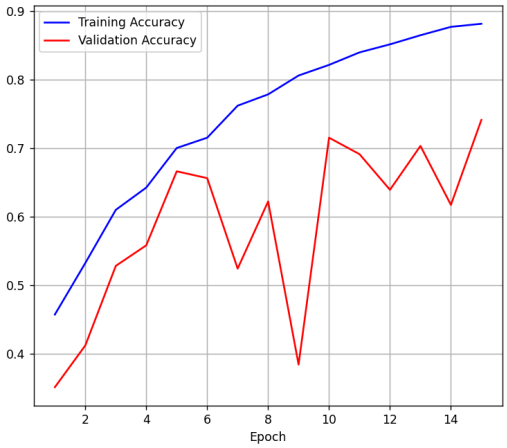
The test accuracy was around 0.60

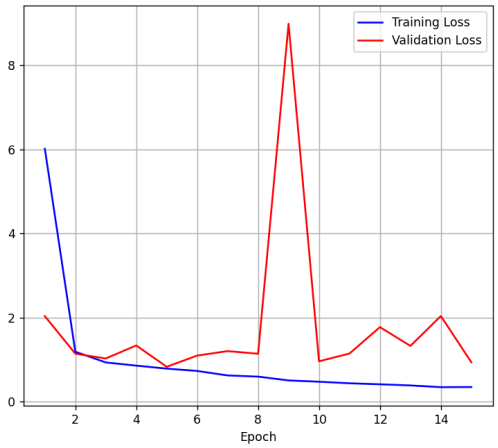
Section 6:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 46, 46, 32)	896
batch_normalization (BN)	(None, 46, 46, 32)	128
max_pooling2d (MaxPool2D)	(None, 23, 23, 32)	0
conv2d_1 (Conv2D)	(None, 21, 21, 64)	18496
batch_normalization_1	(None, 21, 21, 64)	256
max_pooling2d_1	(None, 10, 10, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73856
batch_normalization_2	(None, 8, 8, 128)	512
max_pooling2d_2	(None, 4, 4, 128)	0
flatten	(None, 2048)	0
dense (Dense)	(None, 128)	262272
dropout	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

Total params: 356,803
Trainable params: 356,235
Non-trainable params: 568





In this graph, we can see that the training is not as steady.. For the training and validation accuracy, we see a general increasing trend as epoch increases. Our best model seems to be either epoch 15 with an accuracy above 70%. For the Training loss, we achieved the best at epoch 15 with a loss of 0.84.

Hyperparameter Strategy:

We increased the epoch from 10 to 15 which allowed the model more time to learn giving it higher accuracy. We also added dropouts to prevent overfitting. We kept the learning rate the same as changing it affected the speed and accuracy of the program. We also added a 3rd block to increase model capacity which helped improve training. These changes helped increase the accuracy from above 60% to above 70%.

Section 7: Tic-Tac-Toe:

Example Game:

| | | |

| | | |

| | | |

reference:

row 0 is neutral.

row 1 is happy.

row 2 is surprise.

Turn: X

Player X took position (0, 2).

| | |X|

| | | |

| | | |

Turn: O

Player O took position (0, 0).

|O| |X|

| | | |

| | | |

Turn: X

Player X took position (1, 1).

|O| |X|

| |X| |

| | | |

Turn: O

Player O took position (2, 2).

|O| |X|

| |X| |

| | |O|

Turn: X

Player X took position (2, 0).

|O| |X|

| |X| |

|X| |O|

Game over: X wins!

The interface did work well but seemed to slightly bias a neutral position as it would be a little difficult to get the other two facial expressions.

It seemed like 50/50 between the model thinking it was a neutral face expression versus the other two. Our model graph ~75% accuracy so it seemed like the actual play was worse than the model. It could have been because my laptop's camera is not great

and bad lighting. Also, I have a long beard so possibly the samples did not have as many people with beards?

def _get_emotion(self, img) -> int:

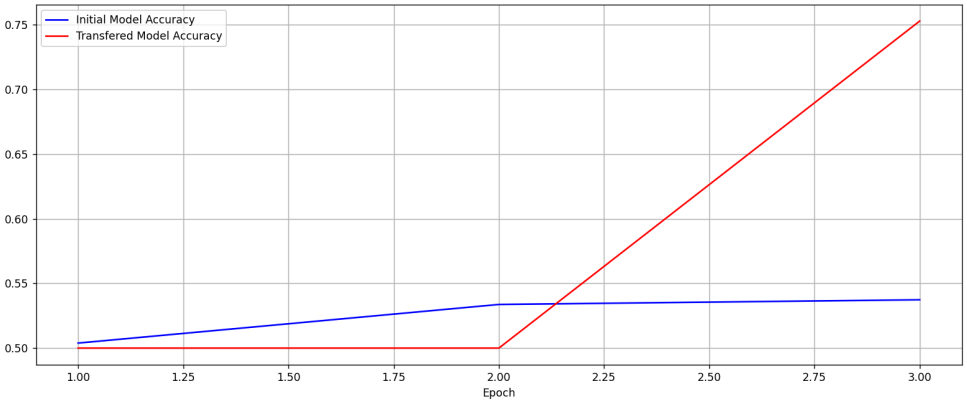
```
    if not hasattr(self, 'model'):
        self.model = load_model("model.keras")


    # Resize the image to match the input size used during training
    img = cv2.resize(img, image_size)
    img = cv2.cvtColor(img, cv2.COLOR_GRAY2RGB)
    img = img.astype("float32") / 255.0
    img = np.expand_dims(img, axis=0)


    # Predict
    predictions = self.model.predict(img)
    predicted_class = int(np.argmax(predictions))


    return predicted_class
```

Section 8



[media pointer="file-service://file-2kXKe6fBkVMMmddpUk8pHz"]
* Data preprocessing
Found 23979 files belonging to 2 classes.
Found 8000 files belonging to 2 classes.
Found 1021 files belonging to 2 classes.
* Training transfered_model for 3 epochs
Model: "model_1"

Layer (type)	Output Shape	Param #
conv2d_input (InputLayer)	[(None, 150, 150, 3)]	0
conv2d (Conv2D)	(None, 148, 148, 32)	896
batch_normalization (BatchNormalization)	(None, 148, 148, 32)	128
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18496
batch_normalization_1 (BatchNormalization)	(None, 72, 72, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73856
batch_normalization_2 (Batc	(None, 34, 34, 128)	512

```
hNormalization)

max_pooling2d_2 (MaxPooling  (None, 17, 17, 128)      0
2D)

flatten (Flatten)          (None, 36992)             0

dense (Dense)              (None, 128)               4735104

dropout (Dropout)          (None, 128)               0

transfer_dense_1 (Dense)    (None, 64)               8256

transfer_output (Dense)     (None, 2)                130

=================================================================
Total params: 4,837,634
Trainable params: 8,386
Non-trainable params: 4,829,248
_________________________________________________________________
Epoch 1/3
188/188 [==============================] - 70s 368ms/step - loss: 7.6196 -
accuracy: 0.4956 - val_loss: 4.0845 - val_accuracy: 0.5039
Epoch 2/3
188/188 [==============================] - 74s 394ms/step - loss: 4.3550 -
accuracy: 0.5106 - val_loss: 2.8096 - val_accuracy: 0.5337
Epoch 3/3
188/188 [==============================] - 74s 392ms/step - loss: 3.2933 -
accuracy: 0.5231 - val_loss: 2.2969 - val_accuracy: 0.5374
* Evaluating transfered_model
8/8 [==============================] - 2s 298ms/step - loss: 2.0452 - accuracy:
0.5867
* Confusion Matrix for transfered_model
8/8 [==============================] - 3s 299ms/step
[[488 247]
 [175 111]]
* Training random_model for 3 epochs
Model: "model_3"
_________________________________________________________________
 Layer (type)              Output Shape              Param #
=================================================================
 conv2d_input (InputLayer)   [(None, 150, 150, 3)]     0

 conv2d (Conv2D)             (None, 148, 148, 32)      896

 batch_normalization (BatchN  (None, 148, 148, 32)     128
```

```
ormalization)

max_pooling2d (MaxPooling2D  (None, 74, 74, 32)       0
)

conv2d_1 (Conv2D)           (None, 72, 72, 64)        18496

batch_normalization_1 (Batc  (None, 72, 72, 64)       256
hNormalization)

max_pooling2d_1 (MaxPooling  (None, 36, 36, 64)       0
2D)

conv2d_2 (Conv2D)           (None, 34, 34, 128)       73856

batch_normalization_2 (Batc  (None, 34, 34, 128)      512
hNormalization)

max_pooling2d_2 (MaxPooling  (None, 17, 17, 128)      0
2D)

flatten (Flatten)           (None, 36992)             0

dense (Dense)               (None, 128)               4735104

dropout (Dropout)           (None, 128)               0

random_dense_1 (Dense)      (None, 64)                8256

random_output (Dense)       (None, 2)                 130

=================================================================
Total params: 4,837,634
Trainable params: 4,837,186
Non-trainable params: 448
_________________________________________________________________
Epoch 1/3
188/188 [==============================] - 265s 1s/step - loss: 1.8287 -
accuracy: 0.5984 - val_loss: nan - val_accuracy: 0.5000
Epoch 2/3
188/188 [==============================] - 271s 1s/step - loss: 1.7378 -
accuracy: 0.6771 - val_loss: nan - val_accuracy: 0.5000
Epoch 3/3
188/188 [==============================] - 281s 1s/step - loss: 1.6563 -
accuracy: 0.7321 - val_loss: 1.6255 - val_accuracy: 0.7530
* Evaluating random_model
```

8/8 [==============================] - 2s 298ms/step - loss: 1.6707 - accuracy: 0.7140
* Confusion Matrix for random_model
8/8 [==============================] - 2s 284ms/step
[[502 233]
 [59 227]]

We used the cats Vs dog dataset from Kaggle.

In the plot, initially the transferred model performed worse than the random model, but significantly improved after the 2nd epoch with an accuracy of 0.75.

However the random model ended up outperforming the transformed model. This is probably due to the transferred model not being fully adjusted.